MIT School of Computing

DepartmentofComputerScienceandEngineering

MIT-ADT UNIVERSITY
PUNE, INDIA
A leap towards World Class Education
(Established by MIT Art, Design and Technology University Act, 2015
(Maharashtra Act No. XXXIX of 2015)

B.Tech: CSE (BTCS) Last Year Year: 2025-26

Deep Learning & Neural Network Lab

Enrolment No.:MITU22BTCS0153 **Division: LY CORE 3**

Roll No.: 2223263 **Name:** Arya Deshmukh

Experiment No 05

Aim:	Write a program to demonstrate the change in accuracy/loss/convergence time with change in optimizers like stochastic gradient descent, adam, adagrad, RMSprop and Nadam for any suitable application
Objectives:	1. To understand the working principles of different gradient-based optimizers. 2. To implement a neural networkmodel using Keras/TensorFlow. 3. To train the same model with different optimizers and compare results. 4. To analyze changes in training accuracy, validation loss, and convergence rate. 5. To visualize results using accuracy and loss plots
Theory	Optimization algorithms are crucial in training neural networks as they determine how model weights are updated during backpropagation to minimize loss. Different optimizers vary in speed, stability, and convergence behavior. 1. Stochastic Gradient Descent (SGD): Updates weights using a small random subset (batch) of data. It is simple and effective but may converge slowly or get stuck in local minima. Update Rule: $$w=w-\eta\cdot\nabla L(w)$$ 2. Adam (Adaptive Moment Estimation): Combines momentum and RMSprop ideas by maintaining moving averages of both gradients and their squares. It converges faster and is widely used. 3. Adagrad: Adapts the learning rate individually for each parameter. Works well for sparse data but learning rate decreases rapidly over time. 4. RMSprop: Improves Adagrad by maintaining an exponentially decaying average of squared gradients. Effective for recurrent and deep networks.

	5. Nadam (Nesterov-accelerated Adam): Combines Adam with Nesterov momentum, providing faster convergence and better generalization in some cases.

Code	import tensorflow as tf
	from tensorflow.keras.datasets import mnist
	from tensorflow.keras.models import Sequential
	from tensorflow.keras.layers import Dense, Flatten
	from tensorflow.keras.utils import to_categorical
	import matplotlib.pyplot as plt
	import time
	
	# Load MNIST dataset
	(x_train, y_train), (x_test, y_test) = mnist.load_data()
	
	# Normalize and preprocess
	x_train = x_train.astype("float32") / 255.0
	x_test = x_test.astype("float32") / 255.0
	y_train = to_categorical(y_train, 10)
	y_test = to_categorical(y_test, 10)
	
	# Define a simple neural network
	def build_model(optimizer):
	  model = Sequential([
	  Flatten(input_shape=(28, 28)),
	  Dense(128, activation="relu"),
	  Dense(64, activation="relu"),
	  Dense(10, activation="softmax")
	  ])
	  model.compile(optimizer=optimizer,
	  loss="categorical_crossentropy",
	  metrics=["accuracy"])
	  return model
	
	# List of optimizers to compare
	optimizers = {

```
 "SGD": tf.keras.optimizers.SGD(),
   "Adam": tf.keras.optimizers.Adam(),
   "Adagrad": tf.keras.optimizers.Adagrad(),
   "RMSprop": tf.keras.optimizers.RMSprop(),
   "Nadam": tf.keras.optimizers.Nadam()
}

# Store results
results = {}

for name, opt in optimizers.items():
   print(f"\nTraining with {name} optimizer...")
   model = build_model(opt)

   start_time = time.time()
   history = model.fit(x_train, y_train,
                       epochs=5, # Keep small for demo
                       batch_size=128,
                       validation_split=0.1,
                       verbose=0)
   end_time = time.time()

   test_loss, test_acc = model.evaluate(x_test, y_test, verbose=0)

   results[name] = {
      "history": history.history,
      "test_acc": test_acc,
      "test_loss": test_loss,
      "time": end_time - start_time
   }
   print(f"{name} -> Test Accuracy: {test_acc:.4f}, Time: {results[name]['time']:.2f} sec")

# Plot accuracy curves
plt.figure(figsize=(12, 5))
for name in results:
   plt.plot(results[name]["history"]["val_accuracy"], label=name)
plt.title("Validation Accuracy per Epoch")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
```

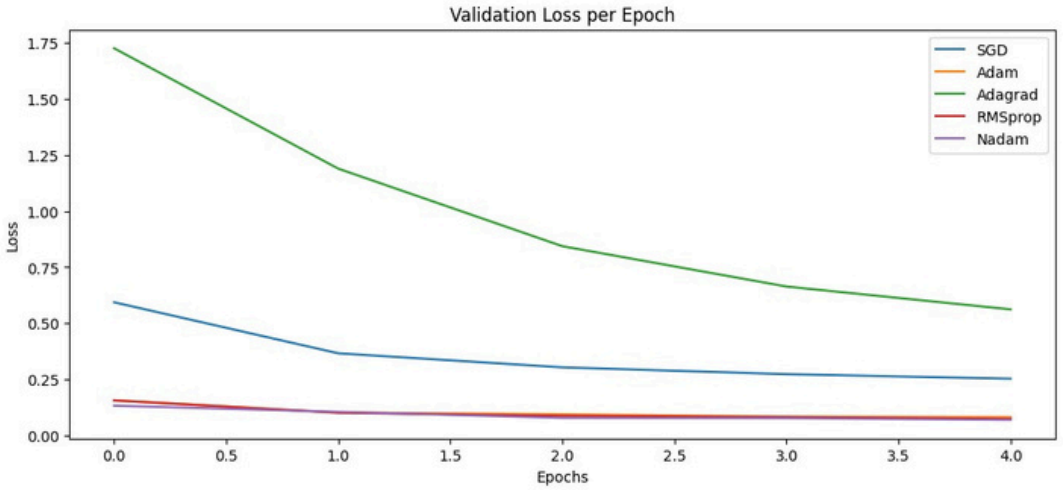
```
plt.legend()
plt.show()

# Plot loss curves
plt.figure(figsize=(12, 5))
for name in results:
    plt.plot(results[name]["history"]["val_loss"], label=name)
plt.title("Validation Loss per Epoch")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()
plt.show()

# Display convergence times
print("\nConvergence Time Comparison:")
forname in results:
    print(f"{name}: {results[name]['time']:.2f} sec")
```

Output (Paste

B.Tech: CSE (BTCS) Last Year Year: 2025-26

Screenshot)	 Convergence Time Comparison: SGD: 14.39 sec Adam: 13.47 sec Adagrad: 13.28 sec RMSprop: 13.16 sec Nadam: 15.80 sec
Conclusion :	This experiment demonstrated how various **optimizers** impact the performance of a neural network. Among all, **Adam** and **Nadam** achieved the best convergence speed and accuracy, while **SGD** provided consistent but slower improvement. Thus, optimizer selection plays a key role in training efficiency and model accuracy

Grade / Marks:	Subject Teacher Signature: Date: